

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

«Харківський авіаційний інститут»



Факультет Інтелектуальних систем управління

Кафедра Систем управління літальними апаратами

Лабораторна робота № 8

з дисципліни «Алгоритмізація та програмування»

на тему: «Реалізація алгоритмів сортування та робота з файлами на мові C ++»

XAI.301.G7.319.21 ПР

Виконав(ла): здобувач(ка) 1 курсу, групи 319
напряму підготовки (спеціальності)

G7 Автоматизація, комп'ютерно-інтегровані
технології та робототехніка

Лесніков А.О.

Прийняла: доц. каф.301, канд. техн. наук,

_____Олена ГАВРИЛЕНКО

МЕТА РОБОТИ

Вивчити теоретичний матеріал по алгоритмам обробки масивів на мові C++, а також бібліотеки для роботи з файлами і реалізувати оголошення, введення з файлу, обробку і виведення в файл одновимірних і двовимірних масивів на мові C++ в середовищі Visual Studio.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. За допомогою текстового редактору створити текстовий файл «array_in_n.txt» з елементами вихідного масиву (n - номер варіанта). У програмі на C++ зчитати і перетворити цей масив відповідно до свого варіанту завдання, ім'я файлу і необхідні змінні ввести з консолі. Вивести результати у файл «array_out_n.txt».

Завдання 2. За допомогою текстового редактору створити текстовий файл «matr_in_n.txt» з елементами вихідного двовимірного масиву (n – номер варіанта). У програмі зчитати і обробити матрицю відповідно до свого варіанту завдання, ім'я файлу і необхідні змінні ввести з консолі. Дописати результати в той же файл.

Завдання 3. Вивчити метод сортування відповідно до свого варіанту, проаналізувати його складність і продемонструвати на прикладі з 7-ми елементів (відповідно до свого варіанту). Реалізувати у вигляді окремої функції алгоритм сортування елементів масиву. Зчитування і виведення відсортованого масиву організувати на файлах.

Завдання 4. Введення, виведення, обробку масивів реалізувати окремими функціями з параметрами. Структурувати проєкт програми для виконання завдань 1-3 наступним чином:

```
main.cpp //основна функція і три функції для 3х завдань
array_utils.h //заголовки функцій для роботи з
//одновимірними масивами (завдання1,3)
array_utils.cpp //визначення функцій для роботи з
//одновимірними масивами (завдання1,3)
matrix_utils.h //заголовки функцій для роботи з
//двовимірними масивами (завдання2)
matrix_utils.cpp //визначення функцій для роботи з
//двовимірними масивами (завдання2)
```

Завдання 5. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань»

Проаналізуйте задані питання, коментарі і оцінки, надані ШІ. Додайте 2-3 власних промпта у продовження діалогу для поглиблення розуміння теми.

ВИКОНАННЯ РОБОТИ

Завдання 1

Реалізувати програму для зчитування одновимірному масиву з файлу, виконати циклічний зсув елементів масиву вліво та записати результат у вихідний файл.

Array84 — циклічний зсув масиву вліво (дійсний, файл)

Вихідний файл

5

3.40 7.50 8.00 -5.50 0.00

Опис алгоритму зчитування та запису:

1. Користувач вводить ім'я вхідного файлу.
2. Програма відкриває файл для читання.
3. Зчитується кількість елементів масиву.
4. У циклі зчитуються всі елементи масиву.
5. Виконується циклічний зсув масиву вліво.

6. Відкривається вихідний файл.

7. Результат записується у файл.

8. Файл закривається

Лістинг коду вирішення задачі наведено в дод. А.1-А.3

В результаті виконання програми масив було циклічно зсунуто вліво.

Початковий масив:

3.40 7.50 8.00 -5.50 0.00

Після зсуву:

7.50 8.00 -5.50 0.00 3.40

Скріншот виконання завдання 1 наведено в дод Б.1

Завдання 2

Реалізувати програму для зчитування матриці з файлу та визначення кількості рядків, схожих на перший рядок матриці.

Вхідний файл

```
4 4
1 2 3 1
3 1 2 3
4 5 6 7
2 3 1 2
```

Опис алгоритму:

1. Відкрити файл з матрицею.
2. Зчитати кількість рядків та стовпців.
3. Зчитати елементи матриці.
4. Порівняти кожний рядок з першим.
5. Підрахувати кількість схожих рядків.
6. Дописати результат у файл.
7. Закрити файл

Програма успішно визначила кількість рядків, схожих на перший.

Кількість схожих рядків:2

Лістинг коду вирішення задачі наведено в дод. А.4-А.5

Скріншот виконання завдання 2 наведено в дод Б.2

Завдання 3

Реалізувати алгоритм шейкерного сортування масиву за спаданням із використанням файлів для введення та виведення даних.

Вхідний файл

7

3.50 1.20 7.80 4.40 2.10 9.00 5.30

Опис алгоритму:

1. Відкрити файл.
2. Зчитати кількість елементів масиву.
3. Зчитати елементи масиву.
4. Виконати шейкерне сортування.
5. Записати відсортований масив у файл.
6. Закрити файл

Результат виконання

Початковий масив:

3.50 1.20 7.80 4.40 2.10 9.00 5.30

Відсортований масив:

9.00 7.80 5.30 4.40 3.50 2.10 1.20

Лістинг коду вирішення задачі наведено в дод. А.1-А.3

Скріншот виконання завдання 3 наведено в дод Б.3

Діаграма активності функції зчитування масиву з файлу

Призначення функції

Функція призначена для відкриття текстового файлу, зчитування кількості елементів масиву та запису елементів у масив програми.

Детальний опис алгоритму

1. Початок роботи функції.

2. Введення імені файлу користувачем.
3. Відкриття файлу для читання.
4. Перевірка успішності відкриття файлу.
5. Якщо файл не відкрився:
 - вивести повідомлення про помилку;
 - завершити функцію.
6. Якщо файл відкрився:
 - зчитати кількість елементів масиву n ;
 - виділити пам'ять для масиву.
7. У циклі зчитати всі елементи масиву.
8. Закрити файл.
9. Передати масив у програму для подальшої обробки.
10. Завершення роботи функції.

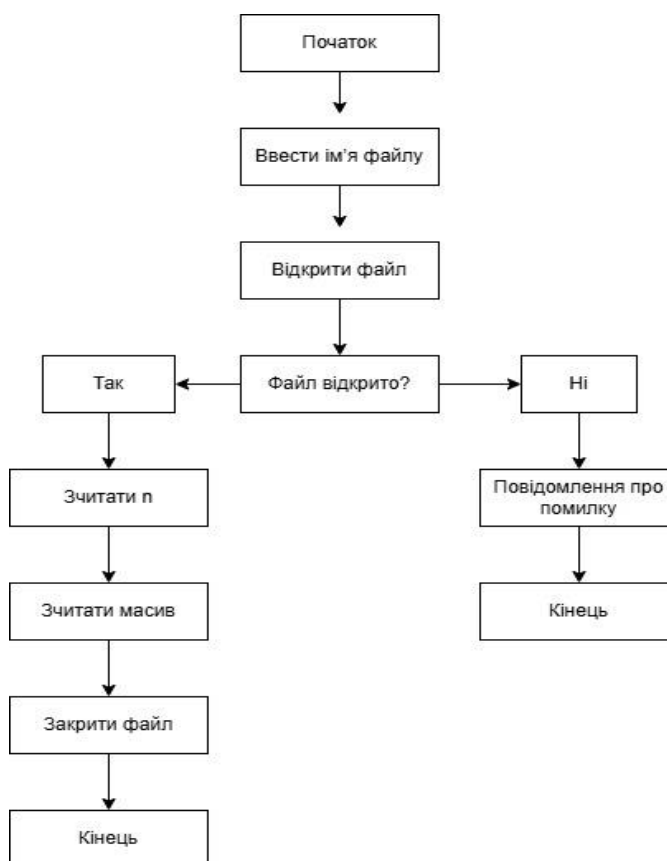


Рисунок 1 – Діаграма активності функції зчитування масиву з файлу

Діаграма активності алгоритму шейкерного сортування

Призначення алгоритму

Алгоритм шейкерного сортування використовується для впорядкування елементів масиву за спаданням.

Детальний опис алгоритму

1. Початок роботи алгоритму.
2. Встановлення меж сортування:
`left = 0;`
`right = n - 1.`
3. Встановлення прапорця `swapped = true.`
4. Початок циклу сортування.
5. Прохід масиву зліва направо:
 - порівнюються сусідні елементи;
 - якщо лівий елемент менший за правий — виконується обмін.
6. Після завершення проходу права межа зменшується.
7. Прохід масиву справа наліво:
 - порівнюються сусідні елементи;
 - якщо правий елемент більший — виконується обмін.
8. Після завершення проходу ліва межа збільшується.
9. Якщо перестановок не було — сортування завершується.
10. Якщо перестановки були — цикл повторюється.
11. Кінець роботи алгоритму.



Рисунок 2 – Діаграма активності алгоритму шейкерного сортування

Покроковий опис виконання алгоритму сортування

I прохід зліва направо

3.50	1.20	7.80	4.40	2.10	9.00	5.30
3.50	7.80	1.20	4.40	2.10	9.00	5.30
3.50	7.80	4.40	1.20	2.10	9.00	5.30
3.50	7.80	4.40	2.10	1.20	9.00	5.30
3.50	7.80	4.40	2.10	9.00	1.20	5.30
3.50	7.80	4.40	2.10	9.00	5.30	1.20

I прохід справа наліво

3.50	7.80	4.40	9.00	2.10	5.30	1.20
3.50	7.80	9.00	4.40	2.10	5.30	1.20
3.50	9.00	7.80	4.40	2.10	5.30	1.20
9.00	3.50	7.80	4.40	2.10	5.30	1.20

II прохід зліва направо

9.00	7.80	3.50	4.40	2.10	5.30	1.20
9.00	7.80	4.40	3.50	2.10	5.30	1.20
9.00	7.80	4.40	3.50	2.10	5.30	1.20
9.00	7.80	4.40	3.50	5.30	2.10	1.20

II прохід справа наліво						
9.00	7.80	4.40	3.50	5.30	2.10	1.20
9.00	7.80	4.40	5.30	3.50	2.10	1.20
9.00	7.80	5.30	4.40	3.50	2.10	1.20
9.00	7.80	5.30	4.40	3.50	2.10	1.20
III прохід зліва направо						
9.00	7.80	5.30	4.40	3.50	2.10	1.20
9.00	7.80	5.30	4.40	3.50	2.10	1.20
9.00	7.80	5.30	4.40	3.50	2.10	1.20

ВИСНОВКИ

У ході виконання лабораторної роботи було реалізовано роботу з файлами, одновимірними та двовимірними масивами у мові C++. Також було освоєно використання алгоритмів сортування та структурування програми на декілька файлів із використанням функцій. У результаті роботи були отримані практичні навички зчитування, обробки та запису даних у файли, а також аналізу роботи алгоритмів сортування.

ДОДАТОК А

Лістинг коду програми

```
// main.cpp
// Лабораторна робота №8 – Сорткування та робота з файлами
// Варіант: Лесніков Артем Олександрович, група 319
// Завдання 1: Array84 – циклічний зсув масиву вліво (дійсний, файл)
// Завдання 2: Matrix36 – кількість рядків, схожих на перший (ціле, файл)
// Завдання 3: Variant20 – шейкерне сорткування за спаданням (дійсний, файл)

#include <iostream>
#include <string>
#include <limits>
#include <stdexcept>
#ifdef _WIN32
#include <windows.h>
#endif
#include "array_utils.h"
#include "matrix_utils.h"
using namespace std;
// -----
// Завдання 1 (Array84): зчитати масив з файлу, виконати циклічний
// зсув вліво, зберегти результат у вихідний файл
// -----
void task1() {
    string in_file, out_file;
    double mas[MAX_N];
    int n;

    cout << "\n=== Завдання 1: Циклічний зсув масиву вліво ===\n";
    cout << "Ім'я вхідного файлу: ";
    cin >> in_file;
    cout << "Ім'я вихідного файлу: ";
    cin >> out_file;

    try {
        // Зчитуємо масив з файлу
        get_mas(in_file, mas, n);

        cout << "Вихідний масив: ";
        print_mas(mas, n);

        // Виконуємо циклічний зсув вліво (Array84)
        cyclic_left_shift(mas, n);

        cout << "Після циклічного зсуву вліво: ";
        print_mas(mas, n);
    }
}
```

```

        // Зберігаємо результат у файл
        show_mas(out_file, mas, n);
        cout << "Результат збережено у файл \"" << out_file << "\"\n";

    } catch (const exception& e) {
        cout << "Помилка: " << e.what() << "\n";
    }
}

// -----
// Завдання 2 (Matrix36): зчитати матрицю з файлу, знайти кількість
// рядків схожих на перший, дописати результат у той самий файл
// -----
void task2() {
    string filename;
    int mx[MAX_M][MAX_NM];
    int m, n;

    cout << "\n=== Завдання 2: Кількість рядків, схожих на перший ===\n";
    cout << "Ім'я файлу матриці: ";
    cin >> filename;

    try {
        // Зчитуємо матрицю з файлу
        get_matrix(filename, mx, m, n);

        cout << "Матриця (" << m << " x " << n << "):\n";
        print_matrix(mx, m, n);

        // Рахуємо рядки, схожі на перший (Matrix36)
        int similar = count_similar_rows(mx, m, n);
        cout << "Кількість рядків, схожих на рядок 1: " << similar << "\n";

        // Допишуємо результат у той самий файл (згідно умови задачі)
        append_result(filename, similar);
        cout << "Результат дописано у файл \"" << filename << "\"\n";

    } catch (const exception& e) {
        cout << "Помилка: " << e.what() << "\n";
    }
}

// -----
// Завдання 3 (Variant 20): зчитати масив з файлу, відсортувати
// шейкерним методом за спаданням, зберегти результат у файл
// -----
void task3() {
    string in_file, out_file;
    double mas[MAX_N];
    int n;

```

```

cout << "\n=== Завдання 3: Шейкерне сортування за спаданням ===\n";
cout << "Ім'я вхідного файлу: ";
cin >> in_file;
cout << "Ім'я вихідного файлу: ";
cin >> out_file;

try {
    // Зчитуємо масив дійсних чисел з файлу
    get_mas(in_file, mas, n);

    cout << "Вихідний масив (" << n << " елементів): ";
    print_mas(mas, n);

    // Виконуємо шейкерне сортування за спаданням
    shaker_sort_desc(mas, n);

    cout << "Відсортований масив (за спаданням): ";
    print_mas(mas, n);

    // Зберігаємо відсортований масив у файл
    show_mas(out_file, mas, n);
    cout << "Відсортований масив збережено у файл \"" << out_file << "\"\n";

} catch (const exception& e) {
    cout << "Помилка: " << e.what() << "\n";
}

}

// -----
// Головна функція – меню вибору завдань
// -----
int main() {
#ifdef _WIN32
    // Встановити UTF-8 для коректного відображення кирилиці в консолі Windows
    SetConsoleOutputCP(65001);
    SetConsoleCP(65001);
#endif
    int choice;

    do {
        cout << "\n===== \n";
        cout << "   ЛР №8 – Сортування та файли\n";
        cout << "===== \n";
        cout << "1. Завдання 1: Циклічний зсув масиву (Array84)\n";
        cout << "2. Завдання 2: Схожі рядки матриці (Matrix36)\n";
        cout << "3. Завдання 3: Шейкерне сортування (Variant20)\n";
        cout << "0. Вихід\n";
        cout << "Ваш вибір: ";

        // Перевірка коректності вводу

```

```

    if (!(cin >> choice)) {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Введіть ціле число!\n";
        choice = -1;
        continue;
    }

    switch (choice) {
        case 1: task1(); break;
        case 2: task2(); break;
        case 3: task3(); break;
        case 0: cout << "До побачення!\n"; break;
        default: cout << "Невірний вибір. Введіть 0-3.\n";
    }

    } while (choice != 0);

    return 0;
}

```

Лістинг А.1 - основна функція і три функції для 3х завдань

```

// array_utils.h
// Заголовки функцій для роботи з одновимірними масивами (завдання 1, 3)
// Варіант: Array84 (циклічний зсув вліво) + Шейкерне сортування за спаданням

#pragma once
#include <string>

const int MAX_N = 20; // максимальний розмір одновимірного масиву

// Зчитати масив дійсних чисел з файлу
// Формат файлу: перший рядок - кількість елементів, другий - елементи через пробіл
void get_mas(const std::string& filename, double in_mas[], int& in_n);

// Записати масив дійсних чисел у файл
void show_mas(const std::string& filename, const double out_mas[], int n);

// Вивести масив у консоль
void print_mas(const double mas[], int n);

// Array84: циклічний зсув масиву вліво на 1 позицію
// A[0]→A[N-1], A[1]→A[0], A[2]→A[1], ..., A[N-1]→A[N-2]
void cyclic_left_shift(double mas[], int n);

// Шейкерне (коктейльне) сортування за спаданням
// Елементи типу double, порядок: від більшого до меншого

```

```
void shaker_sort_desc(double mas[], int n);
```

Лістинг А.2 - заголовки функцій для роботи з одновимірними масивами (завдання1,3)

```
// array_utils.cpp
// Визначення функцій для роботи з одновимірними масивами (завдання 1, 3)

#include "array_utils.h"
#include <iostream>
#include <fstream>
#include <iomanip>
#include <stdexcept>
using namespace std;

// Зчитати масив дійсних чисел з файлу
void get_mas(const string& filename, double in_mas[], int& in_n) {
    ifstream fin(filename);
    if (!fin.is_open())
        throw runtime_error("Не вдалося відкрити файл: " + filename);

    // Зчитуємо кількість елементів
    fin >> in_n;
    if (!fin || in_n < 2 || in_n > MAX_N)
        throw runtime_error("Некоректний розмір масиву (має бути 2.." +
                               to_string(MAX_N) + ")");

    // Зчитуємо елементи
    for (int i = 0; i < in_n; i++) {
        fin >> in_mas[i];
        if (!fin)
            throw runtime_error("Помилка читання елемента " + to_string(i + 1));
    }
    fin.close();
}

// Записати масив дійсних чисел у файл
void show_mas(const string& filename, const double out_mas[], int n) {
    ofstream fout(filename);
    if (!fout.is_open())
        throw runtime_error("Не вдалося відкрити файл для запису: " + filename);

    // Перший рядок – кількість елементів
    fout << n << "\n";
    // Другий рядок – елементи через пробіл
    fout << fixed << setprecision(2);
    for (int i = 0; i < n; i++) {
        fout << out_mas[i];
        if (i < n - 1) fout << " ";
    }
}
```

```

    }
    fout << "\n";
    fout.close();
}

// Вивести масив у консоль
void print_mas(const double mas[], int n) {
    cout << fixed << setprecision(2);
    for (int i = 0; i < n; i++) {
        cout << mas[i];
        if (i < n - 1) cout << " ";
    }
    cout << "\n";
}

// Array84: циклічний зсув вліво на 1 позицію
// Зберігаємо перший елемент, зсуваємо решту вліво, ставимо збережений в кінець
void cyclic_left_shift(double mas[], int n) {
    double first = mas[0];           // зберегти перший елемент
    for (int i = 0; i < n - 1; i++) // кожен елемент замінюємо наступним
        mas[i] = mas[i + 1];
    mas[n - 1] = first;              // перший елемент іде в кінець
}

// Шейкерне сортування за спаданням
// Прохід зліва направо: менший елемент «тоне» вправо
// Прохід справа наліво: більший елемент «спливає» вліво
void shaker_sort_desc(double mas[], int n) {
    int left = 0;           // ліва межа невідсортованої частини
    int right = n - 1;      // права межа невідсортованої частини
    bool swapped;

    do {
        swapped = false;

        // --- Прохід зліва направо ---
        // Якщо поточний елемент менший за наступний – міняємо місцями
        for (int i = left; i < right; i++) {
            if (mas[i] < mas[i + 1]) {
                double tmp = mas[i];
                mas[i] = mas[i + 1];
                mas[i + 1] = tmp;
                swapped = true;
            }
        }
        right--; // після проходу найменший елемент стоїть на right+1 – виключаємо

        if (!swapped) break; // якщо жодної перестановки – масив вже впорядкований
    }
}

```



```

// --- Прохід справа наліво ---
// Якщо поточний елемент більший за попередній – міняємо місцями
for (int i = right; i > left; i--) {
    if (mas[i] > mas[i - 1]) {
        double tmp = mas[i];
        mas[i]      = mas[i - 1];
        mas[i - 1]  = tmp;
        swapped = true;
    }
}
left++; // після проходу найбільший елемент стоїть на left-1 –
виключаємо

} while (swapped);
}

```

Лістинг А.3 - визначення функцій для роботи з одновимірними масивами (завдання1,3)

```

// matrix_utils.h
// Заголовки функцій для роботи з двовимірними масивами (завдання 2)
// Варіант: Matrix36 – кількість рядків, схожих на перший рядок

#pragma once
#include <string>

const int MAX_M    = 20; // максимальна кількість рядків
const int MAX_NM   = 20; // максимальна кількість стовпців
const int MAX_VAL  = 101; // значення елементів: 0..100

// Зчитати цілочисельну матрицю з файлу
// Формат: перший рядок – M N (рядки і стовпці), далі – елементи по рядках
void get_matrix(const std::string& filename,
               int in_mx[][MAX_NM], int& in_m, int& in_n);

// Вивести матрицю у консоль
void print_matrix(const int mx[][MAX_NM], int m, int n);

// Matrix36: знайти кількість рядків (2..M), схожих на перший рядок
// Два рядки «схожі», якщо збігаються множини різних чисел, що в них
зустрічаються
int count_similar_rows(const int mx[][MAX_NM], int m, int n);

// Дописати результат у той самий файл (завдання 2 вимагає дописати, не
перезаписати)
void append_result(const std::string& filename, int similar_count);

```

Лістинг А.4 - заголовки функцій для роботи з двовимірними масивами (завдання2)

```

// matrix_utils.cpp
// Визначення функцій для роботи з двовимірними масивами (завдання 2)

#include "matrix_utils.h"
#include <iostream>
#include <fstream>
#include <iomanip>
#include <cstring>
#include <stdexcept>
using namespace std;

// Зчитати цілочисельну матрицю з файлу
void get_matrix(const string& filename, int in_mx[][MAX_NM],
               int& in_m, int& in_n) {
    ifstream fin(filename);
    if (!fin.is_open())
        throw runtime_error("Не вдалося відкрити файл: " + filename);

    // Перший рядок файлу: кількість рядків і стовпців
    fin >> in_m >> in_n;
    if (!fin || in_m < 2 || in_m > MAX_M || in_n < 2 || in_n > MAX_NM)
        throw runtime_error("Некоректний розмір матриці (має бути 2.." +
                             to_string(MAX_M) + " x 2.." + to_string(MAX_NM) +
                             ")");

    // Зчитуємо елементи по рядках
    for (int i = 0; i < in_m; i++) {
        for (int j = 0; j < in_n; j++) {
            fin >> in_mx[i][j];
            if (!fin)
                throw runtime_error("Помилка читання елемента [" +
                                     to_string(i+1) + "][" + to_string(j+1) +
                                     "]");
            if (in_mx[i][j] < 0 || in_mx[i][j] > 100)
                throw runtime_error("Значення елементів мають бути 0..100");
        }
    }
    fin.close();
}

// Вивести матрицю у консоль
void print_matrix(const int mx[][MAX_NM], int m, int n) {
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++)
            cout << setw(4) << mx[i][j];
        cout << "\n";
    }
}

```

```

// Matrix36: підрахувати кількість рядків (починаючи з 2-го), схожих на перший
// Алгоритм: для кожного рядка будемо булеву маску присутності значень 0..100,
// порівнюємо маску з маскою першого рядка
int count_similar_rows(const int mx[][MAX_NM], int m, int n) {
    // Маска присутності значень першого рядка (індекс = значення елемента)
    bool row0_set[MAX_VAL];
    memset(row0_set, 0, sizeof(row0_set));
    for (int j = 0; j < n; j++)
        row0_set[mx[0][j]] = true;

    int count = 0;
    for (int i = 1; i < m; i++) { // порівнюємо рядки з 2-го по М-й
        // Маска присутності значень i-го рядка
        bool row_set[MAX_VAL];
        memset(row_set, 0, sizeof(row_set));
        for (int j = 0; j < n; j++)
            row_set[mx[i][j]] = true;

        // Порівнюємо дві маски поелементно
        bool similar = true;
        for (int v = 0; v < MAX_VAL; v++) {
            if (row0_set[v] != row_set[v]) {
                similar = false;
                break;
            }
        }
        if (similar) count++;
    }
    return count;
}

// Дописати результат у той самий файл матриці
void append_result(const string& filename, int similar_count) {
    ofstream fout(filename, ios::app); // режим app = дописування в кінець
    if (!fout.is_open())
        throw runtime_error("Не вдалося відкрити файл для дописування: " +
filename);

    fout << "\nКількість рядків, схожих на перший: " << similar_count << "\n";
    fout.close();
}

```

**Лістинг А.5 - визначення функцій для роботи з двовимірними масивами
(завдання2)**

ДОДАТОК Б

Скрін-шоти вікна виконання програми

```

d:\Projects\Lesnikov\LR8>lr8.exe

=====
  ЛР №8 – Сорткування та файли
=====
1. Завдання 1: Циклічний зсув масиву (Array84)
2. Завдання 2: Схожі рядки матриці (Matrix36)
3. Завдання 3: Шейкерне сорткування (Variant20)
0. Вихід
Ваш вибір: 1

=== Завдання 1: Циклічний зсув масиву вліво ===
Ім'я вхідного файлу: array_in_84.txt
Ім'я вихідного файлу: array_out_84.txt
Вихідний масив: 3.40 7.50 8.00 -5.50 0.00
Після циклічного зсуву вліво: 7.50 8.00 -5.50 0.00 3.40
Результат збережено у файл "array_out_84.txt"

=====
  ЛР №8 – Сорткування та файли
=====
1. Завдання 1: Циклічний зсув масиву (Array84)
2. Завдання 2: Схожі рядки матриці (Matrix36)
3. Завдання 3: Шейкерне сорткування (Variant20)
0. Вихід
Ваш вибір:

```

Рисунок Б.1 – Екран виконання програми для вирішення завдання 1

```

=====
  ЛР №8 – Сорткування та файли
=====
1. Завдання 1: Циклічний зсув масиву (Array84)
2. Завдання 2: Схожі рядки матриці (Matrix36)
3. Завдання 3: Шейкерне сорткування (Variant20)
0. Вихід
Ваш вибір: 2

=== Завдання 2: Кількість рядків, схожих на перший ===
Ім'я файлу матриці: matr_in_36.txt
Матриця (4 x 4):
  1   2   3   1
  3   1   2   3
  4   5   6   7
  2   3   1   2
Кількість рядків, схожих на рядок 1: 2
Результат дописано у файл "matr_in_36.txt"

=====
  ЛР №8 – Сорткування та файли
=====
1. Завдання 1: Циклічний зсув масиву (Array84)
2. Завдання 2: Схожі рядки матриці (Matrix36)
3. Завдання 3: Шейкерне сорткування (Variant20)
0. Вихід
Ваш вибір:

```

Рисунок Б.2 – Екран виконання програми для вирішення завдання 2

```

=====
ЛР №8 – Сортування та файли
=====
1. Завдання 1: Циклічний зсув масиву (Array84)
2. Завдання 2: Схожі рядки матриці (Matrix36)
3. Завдання 3: Шейкерне сортування (Variant20)
0. Вихід
Ваш вибір: 3

=== Завдання 3: Шейкерне сортування за спаданням ===
Ім'я вхідного файлу: sort_in_20.txt
Ім'я вихідного файлу: sort_out_20.txt
Вихідний масив (7 елементів): 3.50 1.20 7.80 4.40 2.10 9.00 5.30
Відсортований масив (за спаданням): 9.00 7.80 5.30 4.40 3.50 2.10 1.20
Відсортований масив збережено у файл "sort_out_20.txt"

=====
ЛР №8 – Сортування та файли
=====
1. Завдання 1: Циклічний зсув масиву (Array84)
2. Завдання 2: Схожі рядки матриці (Matrix36)
3. Завдання 3: Шейкерне сортування (Variant20)
0. Вихід
Ваш вибір:

```

Рисунок Б.2 – Екран виконання програми для вирішення завдання 3

ДОДАТОК В

Діалог з ШІ для самоаналізу

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

Відповідь ШІ

Тестові питання

1. Який клас використовується для зчитування даних із файлу у C++?

- A) ofstream
- B) fstream
- C) ifstream
- D) iostream

Правильна відповідь: C

2. Яка складність алгоритму шейкерного сортування у найгіршому випадку?

- A) $O(\log n)$
- B) $O(n)$
- C) $O(n^2)$
- D) $O(n \log n)$

Правильна відповідь: C

3. Для чого використовуються заголовочні файли .h?

- A) Для запуску програми
- B) Для зберігання функцій main()
- C) Для оголошення функцій та структур
- D) Для компіляції програми

Правильна відповідь: C

4. Що відбудеться, якщо файл не відкрився?

- A) Програма автоматично створить файл
- B) Виникне помилка доступу до пам'яті

C) Потрібно виконати перевірку відкриття файлу

D) Програма продовжить роботу без помилок

Правильна відповідь: C

5. Яка основна перевага функцій у програмі?

A) Зменшення кількості змінних

B) Покращення структури та повторного використання коду

C) Прискорення компіляції

D) Автоматичне сортування даних

Правильна відповідь: B

Відкриті питання

1. Поясніть принцип роботи шейкерного сортування.

Шейкерне сортування виконує почергові проходи масивом зліва направо та справа наліво, поступово переміщуючи елементи у правильні позиції.

2. Чому перевірка відкриття файлу є важливою?

Перевірка дозволяє уникнути помилок під час роботи програми у випадку відсутності файлу або неправильного шляху.

3. Які переваги має поділ програми на декілька файлів?

Це покращує структуру проєкту, спрощує пошук помилок та повторне використання коду.

4. Які недоліки має шейкерне сортування?

Алгоритм має квадратичну складність і працює повільно на великих масивах.

5. Для чого використовуються вкладені цикли при роботі з матрицями?

Вкладені цикли дозволяють послідовно обробляти рядки та стовпці матриці.

2) Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ІІІ висока. Обчисли загальну середню оцінку.

Відповідь III

№	Оцінка	Коментар
1	5/5	Відповідь повна та правильна
2	5/5	Правильно пояснено складність алгоритму
3	4/5	Відповідь коротка, але правильна
4	5/5	Добре пояснено необхідність перевірки файлу
5	5/5	Правильно описано переваги функцій
6	4/5	Є загальний опис алгоритму
7	5/5	Логічне пояснення
8	5/5	Повна відповідь
9	4/5	Недоліки описані коротко
10	5/5	Відповідь правильна

Загальна середня оцінка

4.7 / 5

Аналіз результатів

Отримані результати показали достатній рівень знань з теми роботи з файлами, масивами та алгоритмами сортування. Найкраще були засвоєні питання, пов'язані зі структурою програми та роботою з файлами. Частина відповідей потребує більш детального пояснення принципів роботи алгоритмів сортування

3) Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань.

Відповідь III

Додаткові теоретичні відомості

Приклад використання функції для передачі масиву

```
void printArray(double arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
}
```

Приклад динамічного виділення пам'яті

```
int* arr = new int[n];

delete[] arr;
```

Приклад використання функції swap()

```
swap(arr[i], arr[i + 1]);
```

Приклад перевірки відкриття файлу

```
ifstream fin("input.txt");

if (!fin)
{
    cout << "Помилка відкриття файлу";
}
```

Аналіз відповідей та рекомендації

III звернув увагу на теми, які були недостатньо представлені у коді, зокрема:

- динамічне виділення пам'яті;
- передача масивів у функції;
- перевірка відкриття файлів;
- використання стандартних функцій бібліотеки.

Це дозволило краще зрозуміти організацію програм на C++ та принципи структурованого програмування.

Власні промпти

Промпт 1

Поясни різницю між статичними та динамічними масивами у C++.

Промпт 2

Покажи приклад оптимізації алгоритму сортування для великих масивів.

Промпт 3

Поясни, у яких випадках шейкерне сортування працює ефективніше за бульбашкове сортування.